

Design of the Verification Testbench for `uart_tx`

1 Objective

The goal of this testbench is to verify the UART transmitter module `uart_tx.v` located in `src/`. The assignment requires a verification environment that takes into account a key timing fact: the input clock is much faster than the serial baud rate. Because of that, the testbench must validate the design in terms of clock cycles per UART bit, not in terms of approximate real-number delays.

The final implementation is the self-checking file:

```
tb/tb_uart_tx.v
```

2 Relevant DUT Behavior

The DUT is a basic 8N1 UART transmitter:

- 1 start bit with value 0
- 8 data bits transmitted LSB first
- 1 stop bit with value 1

Its timing is controlled by:

```
localparam CLOCKS_PER_BIT = CLK_FREQ / BAUD_RATE;
```

This means every transmitted bit must stay stable for exactly `CLOCKS_PER_BIT` cycles. The DUT also has a few behavioral details that directly shaped the testbench:

- `rst_n` is active low.
- `tx_en` is sampled only in `s_IDLE`.
- The start bit appears one clock after request acceptance.
- `tx_busy` is a registered signal, not a pure decode of the FSM state.
- A short overlapping request while busy is ignored.
- A held-high `tx_en` can be legally accepted again when the DUT returns to `s_IDLE`.

The last two points are easy to confuse, so the testbench treats them as two different corner cases.

3 Why the Testbench Uses Reduced Parameters

The RTL default parameters are:

- CLK_FREQ = 27_000_000
- BAUD_RATE = 115200
- CLOCKS_PER_BIT = 234

Those values are correct, but they are slower to simulate. To keep the same verification idea while reducing runtime, the implemented testbench instantiates the DUT with:

```
localparam integer CLK_FREQ = 100_000;  
localparam integer BAUD_RATE = 1_000;  
localparam integer CLOCKS_PER_BIT = CLK_FREQ / BAUD_RATE;
```

That gives:

$$\text{CLOCKS_PER_BIT} = 100$$

The important point is that the clock is still two orders of magnitude faster than the baud rate, so the assignment's timing condition is preserved. The testbench therefore remains realistic while running faster.

4 General Architecture of the Testbench

The testbench is self-checking and is organized around a small set of reusable tasks.

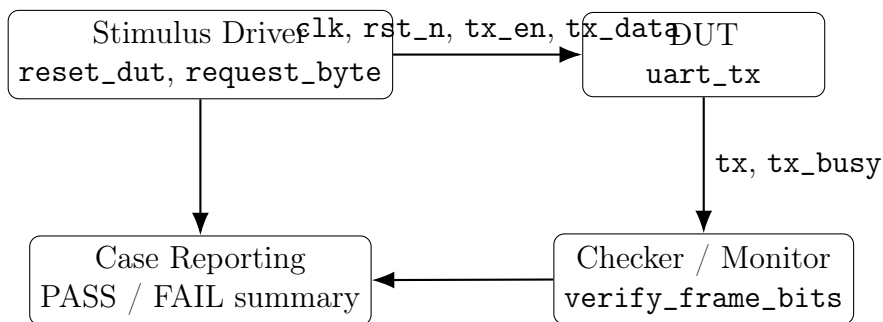


Figure 1: High-level structure of the implemented UART TX testbench.

The main building blocks are:

- a clock generator,
- a reset task,
- a normal one-cycle request driver,
- a short overlapping pulse driver,
- a frame checker based on cycle windows,
- and a case-by-case PASS/FAIL report.

5 Why the Checks Are Cycle-Based

Because the DUT is synchronous, the correct verification reference is the clock. The testbench does not try to approximate UART timing with real-valued delays. Instead, it checks that:

- the start bit begins at the correct clock,
- every bit lasts exactly `CLOCKS_PER_BIT` cycles,
- data is transmitted LSB first,
- and `tx_busy` stays asserted during the active frame.

The key checker is a cycle-window task:

```
task check_bit_window;
    input expected_value;
    input integer bit_kind;
    input integer data_index;
    integer k;
    begin
        for (k = 0; k < CLOCKS_PER_BIT; k = k + 1) begin
            if (tx !== expected_value) begin
                ...
            end
            if (tx_busy !== 1'b1) begin
                ...
            end
            if (k !== CLOCKS_PER_BIT - 1) begin
                @(posedge clk);
                #1;
            end
        end
    end
endtask
```

The small `#1` delay is intentional. The DUT updates `tx` and `tx_busy` with nonblocking assignments on `posedge clk`, so the testbench samples after the edge has settled. This avoids false failures caused by checking the old value in the same simulation time slot.

6 Driver Strategy

Two different request styles are used in the testbench.

Normal Request

The default task applies a one-cycle `tx_en` pulse only when the DUT is idle:

```
task request_byte;
    input [7:0] data_byte;
    output integer accepted_cycle;
```

```

begin
    wait_for_idle();
    @(negedge clk);
    tx_data = data_byte;
    tx_en = 1'b1;
    @(posedge clk);
    #1;
    accepted_cycle = cycle_count;
    ...
    @(negedge clk);
    tx_en = 1'b0;
end
endtask

```

This driver is used for nominal traffic and for back-to-back legal transfers.

Special Request Patterns

Two special patterns are used for corner cases:

- a short overlapping pulse while the DUT is busy,
- a held-high `tx_en` that remains asserted until the DUT returns to `s_IDLE`.

They are both important because the DUT is level-sensitive in `s_IDLE`, not edge-sensitive.

7 Tested Cases

The implemented testbench explicitly highlights and runs the following seven cases.

Case	Purpose
1	Reset behavior
2	Single nominal transmission
3	Pattern sensitivity and LSB-first ordering
4	Data capture vs later <code>tx_data</code> changes
5	Short overlapping request while busy is ignored
6	Held-high <code>tx_en</code> causes legal immediate reacceptance
7	Back-to-back legal transmissions

1. Reset Behavior

This case checks that:

- `tx` immediately goes high during reset,
- `tx_busy` goes low during reset,
- and the DUT remains idle after reset release.

2. Single Nominal Transmission

A non-trivial byte, `8'hA5`, is transmitted and the testbench verifies:

- request acceptance,
- one-cycle delay before the start bit,
- correct frame ordering,
- exact bit duration,
- and return to idle after the stop bit.

3. Pattern Sensitivity and Bit Ordering

Several patterns are sent:

- `8'h00`
- `8'hFF`
- `8'h55`
- `8'hAA`
- `8'h3C`

These patterns give confidence against:

- stuck-at faults,
- wrong bit order,
- bad serialization,
- and missing transitions on the serial output.

4. Data Capture Versus Later Input Changes

The DUT stores the requested byte in `tx_data_reg` when the transfer is accepted. To verify that behavior, the testbench changes `tx_data` during an active frame and checks that the transmitted byte still matches the original accepted value.

5. Short Overlapping Request While Busy

This case applies a one-cycle `tx_en` pulse while a frame is already in progress, then removes it before the DUT returns to idle. The testbench checks that:

- the active frame is not corrupted,
- no extra transfer is accepted,
- and `tx_busy` remains consistent.

6. Held-High `tx_en`

This case is intentionally separate from the previous one. If `tx_en` stays high until the DUT returns to `s_IDLE`, the RTL may legally accept another transfer immediately. The testbench treats this as expected behavior and checks that the second frame starts correctly.

7. Back-to-Back Legal Transmissions

Two valid requests are applied in sequence, each when the DUT is idle. This verifies clean frame termination, correct re-entry into idle, and correct acceptance of the next transfer.

8 Validation Command

The testbench was validated using the repository’s verification builder:

```
./VERIFICATION/builder-verification.sh -r uart-verification -x
```

The build generated:

- the simulation binary in `build/`,
- the waveform file `build/tb_uart_tx.vcd`,
- and a final PASS result from the self-checking testbench.

9 Final Result

The final implemented testbench executed:

- 7 highlighted verification cases,
- 24106 individual checks,
- and 0 failures.

This confirms that the UART transmitter satisfies the intended behavior under reset, nominal transfers, timing-sensitive frame checking, capture semantics, busy overlap handling, held-high request semantics, and back-to-back operation.

10 Conclusion

The final verification strategy is intentionally cycle-accurate, self-checking, and compatible with the repository flow. The design of `tb/tb_uart_tx.v` is driven by the real behavior of the DUT:

- synchronous updates on clock edges,
- bit timing defined by `CLOCKS_PER_BIT`,
- registered `tx_busy`,

- and level-sensitive request handling in `s_IDLE`.

Because of that, the testbench does more than just send bytes and look at the waveform. It actively checks timing windows, handshake semantics, capture behavior, and edge cases that are easy to misinterpret if they are not tested separately.